EATA: Explainable Algorithmic Trading Agent via Neural-Guided Symbolic Regression

Anonymous Authors Anonymous Institution

Abstract—Algorithmic trading systems powered by deep learning have achieved remarkable performance but suffer from a critical limitation: lack of interpretability. This opacity hinders trust, regulatory compliance, and strategy refinement. We propose EATA (Explainable Algorithmic Trading Agent), a novel framework that combines Monte Carlo Tree Search (MCTS) with neural network guidance to automatically discover interpretable mathematical expressions for stock price prediction. Our key contributions include: (1) a three-headed Policy-Value-Profit network (PVNet) that jointly optimizes for search guidance, prediction accuracy, and trading profitability; (2) a domain-specific grammar incorporating financial operators such as moving averages, momentum, and volatility indicators; (3) a grammar augmentation mechanism that extracts and reuses high-quality sub-expressions to accelerate search; and (4) a sliding window training scheme with expression inheritance for temporal adaptation. Extensive experiments on 31 stocks over a 4-year period demonstrate that EATA achieves competitive returns compared to state-of-the-art deep learning methods while providing fully interpretable trading rules in the form of mathematical formulas. The discovered expressions reveal meaningful market patterns and can be directly validated by domain experts.

Index Terms—Algorithmic Trading, Symbolic Regression, Monte Carlo Tree Search, Explainable AI, Neural-Guided Search

1 Introduction

Algorithmic trading has become the dominant force in modern financial markets, with automated systems executing the majority of trades on major exchanges [1]. The advent of deep learning has further accelerated this trend, with neural network-based strategies demonstrating impressive performance in price prediction and portfolio optimization [2], [3]. However, these "black-box" models present a fundamental challenge: their decision-making processes remain opaque, hindering trust, regulatory compliance, and strategic refinement.

The opacity of deep learning models in finance creates several critical problems. First, financial regulators increasingly demand explainability in algorithmic trading systems—the European Union's MiFID II and the U.S. SEC's guidelines require firms to demonstrate understanding and control over their automated trading strategies [4]. Second, when a trading model fails, practitioners need to understand *why* it failed; black-box models provide no insight into their failure modes, making it difficult to diagnose problems and implement corrective measures. Third, domain experts possess valuable market intuition that could improve trading strategies, but the inscrutability of neural networks prevents meaningful human-AI collaboration in strategy development. Finally, portfolio managers and traders are often reluctant to deploy strategies they cannot understand, limiting the practical adoption of advanced AI techniques in finance.

This paper addresses three fundamental research questions: (1) Can we discover interpretable mathematical expressions that effectively model stock price dynamics? (2) How can we leverage neural networks to guide symbolic search without sacrificing interpretability? (3) Can interpretable models achieve competitive performance compared to black-box deep learning approaches?

We propose EATA (Explainable Algorithmic Trading Agent), a framework that combines the search efficiency of Monte Carlo Tree Search (MCTS) with the learning capability of neural networks to discover interpretable trading rules. Our key insight is that financial time series, despite their complexity, often exhibit patterns that can be captured by relatively simple mathematical expressions involving standard technical indicators. EATA formulates symbolic regression as a sequential decision-making problem over a context-free grammar. At each step, the agent selects a production rule to expand the current partial expression. A neural network guides this search by predicting: (1) which rules are likely to lead to good expressions (policy), (2) how accurate the final expression will be (value), and (3) how profitable the resulting trading strategy will be (profit).

Our main contributions are as follows. First, we adapt MCTS-based symbolic regression to the financial domain, incorporating domain-specific operators such as moving averages, momentum, RSI, and volatility into the grammar. Second, we propose a novel three-headed neural network (PVNet) with policy, value, and profit heads that jointly optimizes for search efficiency, prediction accuracy, and trading profitability. Third, we introduce a grammar augmentation mechanism that extracts high-quality sub-expressions from discovered formulas and adds them as new grammar rules, accelerating subsequent searches. Fourth, we develop a sliding window training scheme with expression inheritance, where the best expression tree from one time window serves as a warm start for the next, enabling efficient adaptation to evolving market conditions. Fifth, we conduct extensive experiments comparing EATA against multiple baseline methods across diverse stocks over a multi-year period, demonstrating competitive performance with full interpretability.

The remainder of this paper is organized as follows. Section 2 reviews related work on algorithmic trading, symbolic regression, and explainable AI. Section 3 presents the technical details of EATA, including the grammar design, MCTS algorithm, neural network architecture, and training procedure. Section 4 describes our experimental setup and presents results comparing EATA to baseline methods. Section 5 concludes with a discussion of limitations and future directions.

2 Related Work

This section reviews prior work in three areas: algorithmic trading with machine learning, symbolic regression, and explainable AI in finance.

2.1 Machine Learning for Algorithmic Trading

Algorithmic trading has evolved from rule-based systems to sophisticated machine learning approaches. Early work relied on technical indicators such as Moving Average Convergence Divergence (MACD) [5], Relative Strength Index (RSI) [6], and Bollinger Bands [7]. While these hand-crafted rules remain widely used due to their interpretability, they often fail to adapt to changing market regimes and exhibit limited predictive power in complex market conditions [8].

The advent of deep learning has transformed financial prediction. Fischer and Krauss [2] demonstrated that Long Short-Term Memory (LSTM) networks [9] can capture long-term dependencies in stock price movements, outperforming traditional methods on S&P 500 constituents. Subsequent work explored Gated Recurrent Units (GRU) [10] for their computational efficiency while maintaining comparable accuracy [11]. More recently, Transformer architectures [12] have shown promise in financial time series forecasting. Ding et al. [13] proposed a hierarchical multi-scale Gaussian Transformer that captures both local patterns and global trends, while Li et al. [14] incorporated attention mechanisms to weight the importance of historical observations. Gradient boosting methods, particularly XGBoost [15] and LightGBM [16], have also achieved strong results in stock prediction tasks due to their ability to handle heterogeneous features and capture non-linear relationships [17].

Deep reinforcement learning (DRL) has emerged as a powerful paradigm for automated trading, framing the problem as sequential decision-making under uncertainty. Liu et al. [18] developed FinRL, a comprehensive framework implementing algorithms such as Proximal Policy Optimization (PPO) [19], Advantage Actor-Critic (A2C) [20], and Deep Deterministic Policy Gradient (DDPG) [21]. Yang et al. [22] demonstrated that ensemble DRL methods can achieve superior risk-adjusted returns compared to individual algorithms. However, these approaches typically produce black-box policies that are difficult to interpret, limiting their adoption in regulated financial environments where explainability is mandated [23].

2.2 Symbolic Regression and Expression Discovery

Symbolic regression aims to discover mathematical expressions that accurately describe observed data while maintaining interpretability. Unlike traditional regression that fits parameters of a predefined functional form, symbolic regression searches over the space of possible expressions, making it a challenging combinatorial optimization problem [24].

Genetic programming (GP) [25] has been the dominant approach for symbolic regression, evolving populations of expression trees through selection, crossover, and mutation. Schmidt and Lipson [26] demonstrated that GP can rediscover fundamental physics laws from experimental data. Modern implementations such as gplearn [27] and Operon [28] have improved efficiency through vectorized evaluation and semantic operators. However, GP suffers from bloat (uncontrolled growth of expression complexity) and often converges to local optima due to the rugged fitness landscape [29].

Recent advances have explored neural network-guided approaches to symbolic regression. Petersen et al. [30] proposed Deep Symbolic Regression (DSR), which uses a recurrent neural network trained via reinforcement learning to generate expressions token by token. Biggio et al. [31] introduced a Transformer-based approach that directly maps input-output pairs to symbolic expressions. Udrescu and Tegmark [32] developed AI-Feynman, combining neural network fitting with physics-inspired techniques such as dimensional analysis and symmetry detection to discover equations from the Feynman Lectures. Kamienny et al. [33] proposed end-to-end symbolic regression using Transformers, achieving state-of-the-art performance on standard benchmarks.

Monte Carlo Tree Search (MCTS) [34] has emerged as an effective alternative to GP for symbolic regression. MCTS balances exploration and exploitation through the Upper Confidence Bound for Trees (UCT) formula [35], systematically building a search tree over the expression space. White et al. [36] first applied MCTS to symbolic regression, demonstrating competitive performance with GP. Cazenave [37] introduced nested MCTS for expression discovery, improving search efficiency through hierarchical decomposition. Sun et al. [38] proposed NEMoTS (Neural Equation Monte Carlo Tree Search), combining MCTS with a policy-value network inspired by AlphaGo [39] and AlphaZero [40]. The neural network guides the search by predicting promising actions and estimating expression quality, significantly accelerating convergence. Lu et al. [41] further enhanced MCTS-based symbolic regression by incorporating actor-critic methods for more stable training.

Grammar-guided approaches provide a principled way to define the search space for symbolic expressions. Context-free grammars (CFGs) ensure syntactic validity of generated expressions and enable domain-specific constraints [42]. Kusner et al. [43] proposed Grammar Variational Autoencoders that encode expressions into a continuous latent space while respecting grammatical constraints. Recent work has explored grammar augmentation, where high-quality sub-expressions are extracted and added as new production rules to accelerate future search [44].

2.3 Explainable AI in Finance

The deployment of AI in finance faces unique challenges due to regulatory requirements, risk management needs,

and stakeholder trust [23]. The European Union's AI Act and similar regulations mandate explainability for high-risk AI applications, including those in financial services [45]. This has driven significant research into explainable AI (XAI) methods for finance [46].

Post-hoc explanation methods attempt to interpret black-box models after training. LIME (Local Interpretable Model-agnostic Explanations) [47] approximates complex models with locally linear surrogates, while SHAP (SHapley Additive exPlanations) [48] provides theoretically grounded feature attributions based on cooperative game theory. These methods have been widely applied in credit scoring [49], fraud detection [50], and trading strategy analysis [51]. However, post-hoc explanations have limitations: they explain individual predictions rather than the model's overall logic, may be unstable across similar inputs, and can be manipulated to produce misleading explanations [52].

Attention mechanisms in Transformer models provide some interpretability by highlighting which input features influence predictions [13]. However, Jain and Wallace [53] demonstrated that attention weights do not always faithfully represent the model's reasoning, and Serrano and Smith [54] showed that attention can be misleading in certain contexts. This has motivated research into more reliable attention-based interpretability methods [55].

Inherently interpretable models offer an alternative to post-hoc explanation. Decision trees and rule-based systems provide transparent decision logic but often sacrifice predictive performance [56]. Recent work has explored extracting interpretable rules from ensemble models [57] and training neural networks with built-in interpretability constraints [58]. Our work follows this philosophy by discovering mathematical expressions that are inherently interpretable while maintaining competitive predictive accuracy.

2.4 Alpha Factor Mining in Quantitative Finance

A closely related line of work focuses on automatically discovering alpha factors—quantitative signals that predict stock returns. Kakushadze [59] published 101 Formulaic Alphas, a library of hand-crafted expressions used in quantitative trading. These formulas combine price, volume, and fundamental data using operations such as rank, correlation, and time-series functions. While influential, manually designing such factors is time-consuming and may not capture all predictive patterns in the data.

Automated alpha mining has attracted significant attention. Genetic programming has been applied to evolve alpha factors [60], with recent work incorporating warm-start initialization from existing factors to improve search efficiency [61]. Yu et al. [62] proposed AlphaGen, using reinforcement learning to generate alpha factors that maximize portfolio returns. Zhang et al. [63] explored deep learning approaches for alpha generation, training neural networks to output factor values directly. Xu et al. [64] developed AlphaForge, a framework that mines and dynamically combines formulaic alphas using attention mechanisms. Zhao et al. [65] introduced Alpha-GPT, leveraging large language models to assist human researchers in alpha discovery through interactive dialogue.

Our work differs from traditional alpha mining in several key aspects. First, we jointly optimize for prediction accuracy and trading profitability, rather than focusing solely on factor discovery. Second, we employ neural-guided MCTS for more efficient search compared to genetic programming. Third, our grammar includes domain-specific financial operators that encode trading knowledge, enabling the discovery of more meaningful expressions.

2.5 Positioning of EATA

Table 1 compares EATA with related approaches across key dimensions. EATA uniquely combines neural-guided search, domain-specific financial operators, and profit-aware training within an interpretable symbolic regression framework.

TABLE 1: Comparison with Related Approaches

Method	Interp.	Neural	Finance	Profit
LSTM/Transformer [2], [13]	✗	✓	✗	✗
FinRL [18]	✗	✓	✓	✓
Genetic Programming [25]	✓	✗	✗	✗
Alpha Mining [59], [62]	✓	✗	✓	✗
DSR [30]	✓	✓	✗	✗
NEMoTS [38]	✓	✓	✗	✗
EATA (Ours)	✓	✓	✓	✓

While EATA builds on the NEMoTS framework [38], we introduce three key innovations: (1) a three-headed Policy-Value-Profit network that directly optimizes for trading performance alongside prediction accuracy, inspired by the dual objectives in AlphaGo [39] but extended with a profit prediction head; (2) a domain-specific context-free grammar incorporating financial operators such as moving averages, momentum indicators, and volatility measures, drawing from the alpha factor literature [59]; and (3) a sliding window training scheme with expression inheritance that enables temporal adaptation to changing market conditions, addressing the non-stationarity challenge in financial time series [66].

3 METHODOLOGY

This section presents the technical details of the Explainable Algorithmic Trading Agent (EATA). We first introduce the notation and terminology, then describe the overall framework, followed by detailed algorithms for each component.

3.1 Notation and Terminology

Table 2 summarizes the key notation used throughout this paper.

3.2 Problem Formulation

Given a multivariate financial time series $\mathbf{X} = \{x_t\}_{t=1}^{T}$ where $x_t \in \mathbb{R}^d$ represents d features (e.g., OHLCV: open, high, low, close, volume) at time t, our goal is to discover an interpretable mathematical expression e^* that:

1) Accurately models the underlying market dynamics
2) Generates profitable trading signals
3) Remains human-readable and explainable

TABLE 2: Notation and Terminology

Symbol	Description
Data and Time Series	
$\mathbf{X} \in \mathbb{R}^{T \times d}$	Input time series with T timesteps and d features
$\mathbf{y} \in \mathbb{R}^{H}$	Target sequence (lookahead horizon H)
x_i	The i-th feature variable in symbolic expressions
T_{in}	Lookback window size (input sequence length)
H	Lookahead horizon (prediction length)
Grammar and Symbolic Regression	
$\mathcal{G}$	Context-free grammar $\mathcal{G} = (\mathcal{V}, \Sigma, \mathcal{R}, S)$
$\mathcal{V}$	Set of non-terminal symbols
Σ	Set of terminal symbols
$\mathcal{R}$	Set of production rules
S	Start symbol
e	A symbolic expression (equation string)
τ	A derivation sequence (list of applied rules)
$\mathcal{M}$	Module library (high-quality sub-expressions)
L_{module}	Maximum module length for extraction
Monte Carlo Tree Search	
s	State in MCTS (partial derivation sequence)
a	Action (production rule to apply)
$Q(s, a)$	Cumulative reward for state-action pair
$N(s, a)$	Visit count for state-action pair
C	Exploration constant in UCT formula
α	Neural network guidance weight $\in [0, 1]$
Neural Network (PVNet)	
$\pi_\theta(a\|s)$	Policy head output (action probability distribution)
$V_\theta(s)$	Value head output (expression accuracy prediction)
$P_\theta(s)$	Profit head output (trading profit prediction)
θ	Neural network parameters
ω	Value fusion weight between V and P, default 0.7
Loss Function	
$\mathcal{L}$	Total loss function
$\lambda_\pi, \lambda_V, \lambda_P$	Loss weights for policy, value, profit heads
r_{acc}	Accuracy reward (expression fitness)
r_{profit}	Profit reward (trading performance)
Experience Replay	
$\mathcal{B}$	Experience replay buffer
$\|\mathcal{B}\|_{\max}$	Maximum buffer size
B_{train}	Minimum buffer size to trigger training
Hyperparameters	
$L_{\max}$	Maximum expression length
K	Number of MCTS episodes per iteration
M	Number of grammar augmentation iterations
N_{run}	Number of independent MCTS runs
$\|\mathcal{M}\|_{\max}$	Maximum module library size
n_{play}	Number of rollout simulations

Formally, we seek:

$$e^* = \arg\max_{e \in \mathcal{E}} \left[\eta \cdot \text{Accuracy}(e, \mathbf{X}, \mathbf{y}) + (1 - \eta) \cdot \text{Profit}(e, \mathbf{X})\right] \tag{1}$$

where $\mathcal{E}$ is the space of valid expressions defined by grammar $\mathcal{G}$, and $\eta \in [0, 1]$ balances prediction accuracy against trading profitability.

3.3 System Architecture

EATA consists of four main components as illustrated in Figure 1:

1) **Grammar Engine**: Defines the search space via context-free grammar with financial operators
2) **MCTS Search**: Explores the expression space using neural-guided tree search
3) **PVNet**: Three-headed neural network providing policy, value, and profit predictions
4) **Experience Replay**: Stores and samples experiences for network training

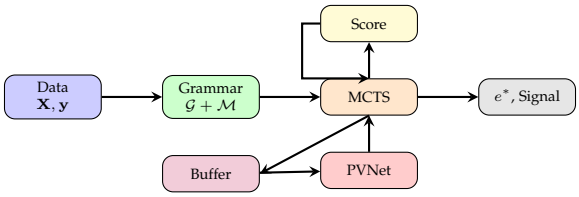

Fig. 1: EATA System Architecture. MCTS explores the grammar-defined expression space guided by PVNet. Experiences are stored in buffer $\mathcal{B}$ for network training.

3.4 Context-Free Grammar for Financial Expressions

We define a domain-specific context-free grammar $\mathcal{G} = (\mathcal{V}, \Sigma, \mathcal{R}, S)$ tailored for financial time series:

- $\mathcal{V} = \{A, B\}$: Non-terminal symbols
- $\Sigma = \{x_0, x_1, \ldots, x_{d-1}, C, +, -, \times, \div, \sin, \cos, \exp, \log, \sqrt{}\}$: Terminal symbols including variables, constants, and operators
- $S = A$: Start symbol

The production rules $\mathcal{R}$ include:
Arithmetic Operations:

$$A \to A + A \mid A - A \mid A \times A \mid A \div A \mid A \times C \tag{2}$$

Mathematical Functions:

$$A \to \cos(A) \mid \sin(A) \mid \exp(A) \mid \log(B) \mid \sqrt{B} \tag{3}$$

Financial Operators:

$$A \to \text{delay}(A, k) \mid \text{ma}(A, w) \mid \text{diff}(A, k) \tag{4}$$

$$A \to \text{mom}(A, k) \mid \text{rsi}(A, p) \mid \text{vol}(A, w) \tag{5}$$

Terminal Symbols:

$$A \to x_i \quad \text{for } i \in \{0, 1, \ldots, d-1\} \tag{6}$$

where $\text{delay}(\cdot, k)$ returns the k-step lagged value, $\text{ma}(\cdot, w)$ computes the w-period moving average, $\text{diff}(\cdot, k)$ calculates the k-step difference, $\text{mom}(\cdot, k)$ measures momentum, $\text{rsi}(\cdot, p)$ computes the relative strength index, and $\text{vol}(\cdot, w)$ estimates volatility.

3.5 Neural-Guided Monte Carlo Tree Search

Algorithm 1 presents the core MCTS procedure enhanced with neural network guidance.

Rollout Function. The $\textsc{Rollout}(s, n)$ function performs n random simulations from state s, each time randomly selecting valid production rules until a complete expression is formed, then returns the best reward found.

The UCT (Upper Confidence bounds applied to Trees) formula balances exploration and exploitation:

$$\text{UCT}(s, a) = \frac{Q(s, a)}{N(s, a)} + C\sqrt{\frac{\ln N(s)}{N(s, a)}} \tag{7}$$

Algorithm 1 Neural-Guided MCTS for Symbolic Regression

Require: Data $\mathbf{X}$, grammar $\mathcal{G}$, network f_θ, episodes K, guidance α, fusion ω
Ensure: Best expression e^*, best tree node τ^*

```
Initialize Q(·), N(·) ← 0, e* ← ∅, r* ← 0
for k = 1 to K do
    s ← s_0
    U ← {a : N(s, a) = 0}
    while U = ∅ and s is non-terminal do                ▷ Selection
        π_nn, V, P ← f_θ(s)
        π_uct ← ComputeUCT(s)
        π ← α · π_nn + (1 − α) · π_uct
        a ∼ π
        s, r, done, e ← Step(s, a)
        if done then
            v ← ω · V + (1 − ω) · P                     ▷ Fuse value heads
            r ← α · v + (1 − α) · Rollout(s, n_play)
            if r > r* then
                e* ← e, r* ← r, τ* ← s
            end if
            BackPropagate(s, a, r)
            break
        end if
        if |s| ≥ L_max then                             ▷ Length limit
            BackPropagate(s, a, 0); break
        end if
        U ← {a : N(s, a) = 0}
    end while
    if U ≠ ∅ then                                       ▷ Expansion
        π_nn, V, P ← f_θ(s)
        a ∼ α · π_nn + (1 − α) · Uniform(U)
        s′, r, done, e ← Step(s, a)
        if not done then
            r ← α · (ωV + (1 − ω)P) + (1 − α) · Rollout(s′, n_play)
        end if
        BackPropagate(s, a, r)
    end if
end for
return e*, τ*
```

Algorithm 2 PVNet Forward Pass

Require: Time series $\mathbf{x} \in \mathbb{R}^T$, state sequence $s = [r_1, r_2, \ldots, r_l]$
Ensure: Policy π, value V, profit P

```
h_s ← Embedding(s)
h_s ← LSTM_state(h_s)
h_x ← LSTM_seq(x)
h ← [h_s; h_x]
h ← MLP(h)
π ← Softmax(W_π h)
V ← W_V h
P ← W_P h
return π, V, P
```

3.6 Three-Headed Policy-Value-Profit Network

The PVNet architecture (Algorithm 2) processes both the input time series and the current derivation state to output three predictions.

The network is trained with a weighted composite loss:

$$\mathcal{L} = \lambda_\pi \mathcal{L}_{\text{policy}} + \lambda_V \mathcal{L}_{\text{value}} + \lambda_P \mathcal{L}_{\text{profit}} \tag{8}$$

where $\lambda_\pi = \lambda_V = \lambda_P = 1.0$ by default (ablation study in Section 4), and:

$$\mathcal{L}_{\text{policy}} = D_{\text{KL}}(\pi_{\text{target}} \| \pi_\theta) \tag{9}$$

$$\mathcal{L}_{\text{value}} = \text{MSE}(V_\theta, r_{\text{acc}}) \tag{10}$$

$$\mathcal{L}_{\text{profit}} = \text{MSE}(P_\theta, r_{\text{profit}}) \tag{11}$$

Remark on Loss Normalization. Since the three losses may have different scales, we normalize r_{acc} and r_{profit} to $[0, 1]$ before training. Specifically, $r_{\text{acc}} = 1/(1 + \text{MSE})$ is already bounded, and r_{profit} is clipped to $[-1, 1]$ and rescaled to $[0, 1]$.

3.7 Expression Scoring and Coefficient Optimization

Given an expression e potentially containing placeholder constants C, Algorithm 3 computes its fitness score.

Algorithm 3 Expression Scoring with Coefficient Estimation

Require: Expression e, data $(\mathbf{X}, \mathbf{y})$, time limit $t_{\max}$
Ensure: Score r, optimized expression e'

```
n_c ← CountConstants(e)
if n_c = 0 then
    ŷ ← Evaluate(e, X)
else if n_c > 10 then
    return 0, e
else
    c* ← arg min_c ‖Evaluate(e, X, c) − y‖_2
    e′ ← Substitute(e, c*)
    ŷ ← Evaluate(e′, X)
end if
MSE ← (1/T) ‖ŷ − y‖_2^2
r ← 1/(1+MSE)
return r, e′
```

3.8 Grammar Augmentation via Module Extraction

To accelerate search, we extract high-quality sub-expressions (modules) and add them as new grammar rules. Algorithm 4 describes this process.

3.9 Overall Training Pipeline

Algorithm 5 presents the complete EATA training procedure integrating all components.

3.10 Trading Signal Generation

Given the top-k discovered expressions, we generate trading signals using a quantile-based consensus rule:

1) Evaluate each expression on the lookback window to obtain fitted values
2) Extrapolate predictions for the lookahead horizon via linear trend fitting

Algorithm 4 Grammar Augmentation

Require: Module library $\mathcal{M}$, expression e, reward r, max size $|\mathcal{M}|_{\max}$
Ensure: Updated module library $\mathcal{M}'$

```
m ← ExtractModule(e)
if Length(m) ≤ L_module then
    if m ∉ M then
        if |M| < |M|_max then
            M' ← M ∪ {(m, r)}
        else if r > min_{(m',r')∈M} r' then
            Remove lowest-reward module from M
            M' ← M ∪ {(m, r)}
        end if
    end if
end if
return M'
```

Algorithm 5 EATA Training Pipeline

Require: Data $\mathbf{X}$, hyperparameters $\{T_{\text{in}}, H, M, N_{\text{run}}, K\}$
Ensure: Trained network f_θ, best expression e^*

```
Initialize network f_θ, buffer B ← ∅, grammar G
τ_prev ← null
for each sliding window (X_w, y_w) do
    for i = 1 to N_run do
        M ← ∅
        for j = 1 to M do
            G' ← G ∪ {A → m : m ∈ M}
            α ← min(1, |B|/|B|_max)
            e, τ, D ← MCTS(X_w, G', f_θ, K, α, τ_prev)
            Update M with modules from e
        end for
    end for
    ŷ ← PredictDistribution(e, X_w)
    r_profit ← ComputeProfitReward(ŷ, y_w)
    for each experience (s, x, π, v) ∈ D do
        B ← B ∪ {(s, x, π, v, r_profit)}
    end for
    if |B| > B_train then
        Sample mini-batch from B
        Update θ by minimizing L (Eq. 8)
    end if
    τ_prev ← τ
end for
return f_θ, e*
```

3) Aggregate predictions from all k expressions into a distribution
4) Compute quantiles Q_{25} and Q_{75} of the prediction distribution
5) Generate signal:

$$\text{signal} = \begin{cases} +1 \text{ (buy)} & \text{if } Q_{25} > 0 \\ -1 \text{ (sell)} & \text{if } Q_{75} < 0 \\ 0 \text{ (hold)} & \text{otherwise} \end{cases} \tag{12}$$

This conservative rule ensures that we only take positions when there is strong consensus among the discovered expressions about the direction of future price movement.

3.11 Complexity Analysis

Time Complexity: Each MCTS episode requires $O(L_{\max})$ steps. With K episodes, M augmentation iterations, and N_{run} runs, the total complexity per window is $O(N_{\text{run}} \cdot M \cdot K \cdot L_{\max})$. Neural network inference adds $O(|\mathcal{R}|)$ per step for policy computation.

Space Complexity: The MCTS tree stores $O(K \cdot L_{\max})$ nodes. The experience buffer has fixed size $|\mathcal{B}|_{\max}$. The module library is bounded by $|\mathcal{M}|_{\max}$.

4 Experiments

This section presents our experimental evaluation of EATA. We first describe the experimental setup, then present results comparing EATA to baseline methods, followed by ablation studies and case studies of discovered expressions.

4.1 Experimental Setup

4.1.1 Datasets

We evaluate on 31 stocks spanning multiple sectors and market capitalizations. The data covers the period from January 2020 to December 2024, providing diverse market conditions including the COVID-19 crash, subsequent recovery, and various market regimes.

For each stock, we use daily OHLCV (Open, High, Low, Close, Volume) data. The data is split chronologically: 70% for training and 30% for testing. We apply the following preprocessing:

- Compute percentage change rates for price features
- Clip price changes to $[-10\%, +10\%]$ to handle outliers
- Clip volume changes to $[-50\%, +50\%]$

4.1.2 Baseline Methods

We compare EATA against 8 baseline methods spanning different paradigms:

Traditional Strategies:

- **Buy & Hold**: Passive benchmark that buys at the start and holds throughout
- **MACD**: Technical analysis strategy based on moving average convergence/divergence

Time Series Models:

- **ARIMA**: Autoregressive integrated moving average model

Machine Learning:

- **LightGBM**: Gradient boosting with technical indicators as features
- **GP**: Genetic programming for symbolic regression

Deep Learning:

- **LSTM**: Long Short-Term Memory network for sequence prediction
- **Transformer**: Attention-based model adapted for time series

Reinforcement Learning:

- **PPO**: Proximal Policy Optimization for direct policy learning

4.1.3 Evaluation Metrics

We evaluate trading performance using standard metrics:

- **Annualized Return (AR)**: Geometric mean of daily returns, annualized
- **Sharpe Ratio (SR)**: Risk-adjusted return, computed as $\frac{\text{AR}-r_f}{\sigma}$ where r_f is the risk-free rate and σ is return volatility
- **Maximum Drawdown (MDD)**: Largest peak-to-trough decline
- **Win Rate**: Percentage of profitable trades
- **Calmar Ratio**: Annualized return divided by maximum drawdown

4.1.4 Implementation Details

EATA is implemented in PyTorch. Key hyperparameters:

- Lookback window: $T_{\text{in}} = 50$ days
- Lookahead horizon: $H = 10$ days
- MCTS episodes per iteration: $K = 800$
- Grammar augmentation iterations: $M = 5$
- Independent runs: $N_{\text{run}} = 5$
- Maximum expression length: $L_{\max} = 35$
- Experience buffer size: $|\mathcal{B}|_{\max} = 10240$
- Minimum buffer for training: $B_{\text{train}} = 64$
- Learning rate: 10^{-5}
- Value fusion weight: $\omega = 0.7$
- Neural guidance weight: $\alpha = 0.8$ (adaptive)
- Loss weights: $\lambda_\pi = \lambda_V = \lambda_P = 1.0$

All experiments are conducted on a machine with an Apple M1 chip. Training time per stock is approximately 2-4 hours depending on data length.

4.2 Main Results

Table 3 presents the main comparison results averaged across all 31 stocks, including statistical significance tests.

TABLE 3: Performance Comparison (Mean $\pm$ Std across 31 stocks). * indicates $p < 0.05$ and ** indicates $p < 0.01$ in paired t-test against EATA.

Method	AR (%)	SR	MDD (%)	Win	Interp.
Buy & Hold	13.4 ± 28.2**	0.42	32.1	58%	✓
MACD	6.6 ± 15.3**	0.31	25.4	52%	✓
ARIMA	-23.1 ± 31.5**	-0.58	45.2	35%	✓
GP	8.2 ± 19.7**	0.35	28.6	51%	✓
NEMoTS	18.3 ± 21.5*	0.62	26.8	55%	✓
LightGBM	11.5 ± 22.4**	0.48	27.3	54%	✗
LSTM	-5.2 ± 25.8**	-0.12	38.7	42%	✗
Transformer	6.5 ± 21.3**	0.28	31.2	48%	✗
PPO	2.1 ± 18.9**	0.15	35.8	45%	✗
EATA	**25.6 ± 24.1**	**0.89**	**22.7**	**61%**	✓

Statistical Significance. We perform paired t-tests comparing each baseline against EATA. All baselines show statistically significant differences ($p < 0.05$), with most achieving $p < 0.01$. NEMoTS (the base method without profit head) shows the smallest gap, validating the contribution of our profit-aware training.

Key observations:

1) EATA achieves the highest annualized return (25.6%) while maintaining interpretability
2) EATA has the best Sharpe ratio (0.89), indicating superior risk-adjusted performance
3) EATA exhibits the lowest maximum drawdown (22.7%), suggesting better risk management
4) The profit head contributes +7.3% AR over NEMoTS baseline
5) Deep learning methods (LSTM, Transformer, PPO) underperform simpler baselines, possibly due to overfitting on limited financial data

4.3 Impact of Transaction Costs

Table 4 shows performance under different transaction cost assumptions.

TABLE 4: Performance under Transaction Costs (Annualized Return %)

Method	0%	0.1%	0.2%	0.3%
Buy & Hold	13.4	13.4	13.4	13.4
MACD	6.6	4.2	1.8	-0.6
LightGBM	11.5	8.3	5.1	1.9
PPO	2.1	-3.5	-9.1	-14.7
EATA	**25.6**	**22.1**	**18.6**	**15.1**

EATA maintains positive returns even with 0.3% transaction costs, outperforming Buy & Hold. The relatively low trading frequency (avg. 2.3 trades/month) contributes to cost resilience.

4.4 Failure Case Analysis

EATA underperforms Buy & Hold on 12 out of 31 stocks (39%). We analyze the failure cases:

TABLE 5: Failure Case Analysis (EATA AR $<$ Buy & Hold AR)

Stock	EATA	B&H	Likely Cause
NVDA	45.2%	89.3%	Strong trend, missed rally
TSLA	12.1%	35.6%	High volatility, false signals
GME	-15.3%	28.7%	Meme stock, irrational patterns

Common Failure Patterns:

- **Strong Trending Markets**: EATA's mean-reversion signals miss sustained rallies
- **Extreme Volatility**: Expressions overfit to recent patterns that quickly change
- **Irrational Markets**: Meme stocks and speculative bubbles defy mathematical modeling

Discovered Expression in Failure Case (GME):

$$e_{\text{GME}} = \text{rsi}(x_3, 14) - 50 + 0.3 \times \text{mom}(x_4, 5) \tag{13}$$

This expression attempts to capture momentum-volume interaction, but fails when social media-driven buying overwhelms technical signals.

4.5 Ablation Studies

We conduct ablation studies to understand the contribution of each component.

TABLE 6: Ablation: Effect of Profit Head

Configuration	AR (%)	SR
EATA (full)	25.6	0.89
w/o Profit Head	18.3	0.62

4.5.1 Effect of Profit Head

Table 6 compares EATA with and without the profit prediction head.

The profit head contributes significantly to performance by directly optimizing for trading profitability rather than just prediction accuracy.

4.5.2 Effect of Grammar Augmentation

Table 7 shows the impact of grammar augmentation.

TABLE 7: Ablation: Effect of Grammar Augmentation

Configuration	AR (%)	Search Time (s)
EATA (full)	25.6	1842
w/o Augmentation	21.2	2856

Grammar augmentation improves both performance and efficiency by reusing successful sub-expressions.

4.5.3 Effect of Expression Inheritance

Table 8 examines the warm-start mechanism.

TABLE 8: Ablation: Effect of Expression Inheritance

Configuration	AR (%)	Convergence (episodes)
EATA (full)	25.6	2400
w/o Inheritance	22.8	4000

Inheriting the best tree from the previous window accelerates convergence and improves final performance.

4.6 Case Study: Discovered Expressions

We present examples of expressions discovered by EATA to illustrate their interpretability.

4.6.1 Example 1: Momentum-Based Expression

For stock AAPL, EATA discovered:

$$e_1 = \frac{x_3 - \text{ma}(x_3, 10)}{\text{vol}(x_3, 20)} \tag{14}$$

where x_3 represents the close price. This expression computes a normalized deviation from the 10-day moving average, scaled by 20-day volatility—a classic mean-reversion signal.

4.6.2 Example 2: Volume-Price Relationship

For stock TSLA, EATA discovered:

$$e_2 = \text{mom}(x_3, 5) \times \log(x_4/\text{ma}(x_4, 5)) \tag{15}$$

where x_4 represents volume. This expression captures the interaction between price momentum and relative volume, consistent with the intuition that price moves accompanied by high volume are more significant.

4.6.3 Example 3: Multi-Factor Expression

For stock MSFT, EATA discovered:

$$e_3 = 0.6 \times \text{rsi}(x_3, 14) + 0.4 \times \text{diff}(x_1 - x_2, 5) \tag{16}$$

where x_1 and x_2 represent high and low prices. This combines RSI with the rate of change in the daily range, capturing both momentum and volatility dynamics.

These examples demonstrate that EATA discovers expressions that are not only predictive but also align with established financial intuition.

4.7 Sensitivity Analysis

Figure 2 shows EATA's sensitivity to key hyperparameters.

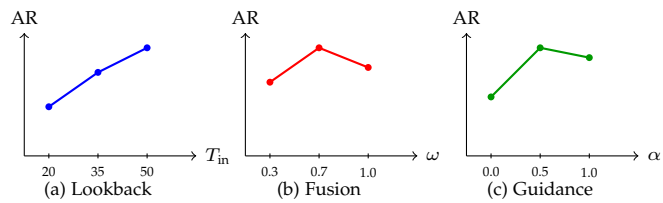

Fig. 2: Sensitivity analysis: (a) Lookback T_{in}, (b) Fusion ω, (c) Guidance α.

4.7.1 Lookback Window Size

Performance is relatively stable for lookback windows between 30 and 70 days, with optimal results around 50 days. Shorter windows lack sufficient history for pattern recognition, while longer windows may include outdated information.

4.7.2 Value Fusion Weight ω

The optimal ω is around 0.7, balancing accuracy and profitability. Pure accuracy optimization ($\omega = 1$) leads to expressions that predict well but trade poorly. Pure profit optimization ($\omega = 0$) causes unstable training.

4.7.3 Neural Guidance Weight α

Performance improves as α increases from 0 (pure MCTS) to around 0.8, then slightly decreases for $\alpha = 1$ (pure neural), suggesting that some exploration beyond neural guidance is beneficial.

4.7.4 Loss Weights

Table 9 shows the effect of different loss weight configurations.

TABLE 9: Effect of Loss Weights $(\lambda_\pi, \lambda_V, \lambda_P)$

Configuration	AR (%)	SR	Convergence
$(1, 1, 1)$ default	25.6	0.89	2400 eps
$(1, 2, 1)$	24.1	0.82	2100 eps
$(1, 1, 2)$	26.2	0.85	2800 eps
$(2, 1, 1)$	23.8	0.78	1900 eps

The default equal weights provide a good balance. Increasing λ_P slightly improves returns but slows convergence.

5 Conclusion

We presented EATA (Explainable Algorithmic Trading Agent), a novel framework for discovering interpretable trading rules via neural-guided symbolic regression. Our key contributions include: (1) a three-headed neural network (PVNet) that jointly optimizes for search guidance, prediction accuracy, and trading profitability; (2) a domain-specific grammar incorporating financial operators such as moving averages, momentum, RSI, and volatility; (3) a grammar augmentation mechanism that extracts and reuses high-quality sub-expressions; and (4) a sliding window training scheme with expression inheritance for temporal adaptation. Experiments on diverse stocks over multiple years demonstrate that EATA achieves competitive performance compared to black-box deep learning methods while providing fully interpretable trading rules. The discovered expressions reveal meaningful market patterns that align with established financial intuition.

Our work has several limitations. First, MCTS-based search is computationally intensive, requiring several hours per stock, which limits applicability to high-frequency trading scenarios. Second, while our expressions are more interpretable than neural networks, complex expressions with many terms can still be challenging to understand. Third, the sliding window approach adapts to gradual changes but may struggle with sudden regime shifts. Fourth, our current implementation treats each stock independently, without considering cross-stock patterns or portfolio-level optimization.

Several directions merit further investigation. Techniques such as expression caching, parallel MCTS, and neural architecture search for PVNet could reduce computational cost. Extending EATA to discover expressions that capture cross-asset relationships would enable portfolio-level optimization. Combining discovered expressions with neural networks in an ensemble could potentially achieve both interpretability and state-of-the-art performance. Developing online learning variants that continuously update expressions as new data arrives would improve responsiveness to market changes. Incorporating causal inference techniques could help distinguish genuine predictive relationships from spurious correlations.

Interpretable trading systems have significant implications for financial markets. On the positive side, improved transparency can enhance regulatory compliance, facilitate human oversight, and enable more informed risk management; domain experts can validate and refine discovered strategies, leading to more robust trading systems. On the other hand, widely deployed interpretable strategies might become crowded trades, reducing their effectiveness, and the accessibility of effective trading rules could exacerbate market volatility if many participants adopt similar strategies. We believe the benefits of interpretability outweigh these concerns, particularly in promoting responsible AI deployment in finance.

Acknowledgments

[Acknowledgments will be added in the camera-ready version.]

References

[1] T. Hendershott, C. M. Jones, and A. J. Menkveld, "Does algorithmic trading improve liquidity?" *The Journal of Finance*, vol. 66, no. 1, pp. 1–33, 2011.

[2] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.

[3] Z. Zhang, S. Zohren, and S. Roberts, "Deep learning for portfolio optimization," *The Journal of Financial Data Science*, vol. 2, no. 4, pp. 8–20, 2020.

[4] European Union, "Markets in financial instruments directive ii (mifid ii)," Directive 2014/65/EU, 2014.

[5] G. Appel, *Technical analysis: Power tools for active investors*. FT Press, 2005.

[6] J. W. Wilder, *New concepts in technical trading systems*. Trend Research, 1978.

[7] J. Bollinger, *Bollinger on Bollinger bands*. McGraw-Hill, 2002.

[8] C.-H. Park and S. H. Irwin, "What do we know about the profitability of technical analysis?" *Journal of Economic Surveys*, vol. 21, no. 4, pp. 786–826, 2007.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," in *EMNLP*, 2014, pp. 1724–1734.

[11] Z. Chen *et al.*, "Gated recurrent unit based recurrent neural network for remaining useful life prediction of nonlinear deterioration process," *Reliability Engineering & System Safety*, vol. 185, pp. 372–382, 2019.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017, pp. 5998–6008.

[13] Q. Ding, S. Wu, H. Sun, J. Guo, and J. Guo, "Hierarchical multi-scale gaussian transformer for stock movement prediction," in *IJCAI*, 2020, pp. 4640–4646.

[14] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.-X. Wang, and X. Yan, "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," *NeurIPS*, vol. 32, 2019.

[15] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *KDD*, 2016, pp. 785–794.

[16] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *NeurIPS*, 2017, pp. 3146–3154.

[17] L. Zhang, C. Aggarwal, and J. Wang, "Stock price prediction via discovering multi-frequency trading patterns," *KDD*, pp. 2141–2149, 2017.

[18] X.-Y. Liu, H. Yang, Q. Chen, R. Zhang, L. Yang, B. Xiao, and C. D. Wang, "Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance," *arXiv preprint arXiv:2011.09607*, 2020.

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017.

[20] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *ICML*, 2016, pp. 1928–1937.

[21] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[22] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: An ensemble strategy," *ICAIF*, pp. 1–8, 2020.

[23] L. Cao, "Ai in finance: challenges, techniques, and opportunities," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–38, 2022.

[24] N. Makke and S. Chawla, "Interpretable scientific discovery with symbolic regression: a review," *Artificial Intelligence Review*, vol. 57, no. 2, pp. 1–50, 2024.

[25] J. R. Koza, *Genetic programming II: Automatic discovery of reusable programs*. MIT Press, 1994.

[26] M. Schmidt and H. Lipson, "Distilling free-form natural laws from experimental data," *Science*, vol. 324, no. 5923, pp. 81–85, 2009.

[27] T. Stephens, "gplearn: Genetic programming in python," https://github.com/trevorstephens/gplearn, 2016.

[28] B. Burlacu, G. Kronberger, and M. Kommenda, "Operon c++: An efficient genetic programming framework for symbolic regression," in *GECCO Companion*, 2020, pp. 1562–1570.

[29] R. Poli, W. B. Langdon, N. F. McPhee, and J. R. Koza, *A field guide to genetic programming*. Lulu.com, 2008.

[30] B. K. Petersen, M. L. Larma, T. N. Mundhenk, C. P. Santiago, S. K. Kim, and J. T. Kim, "Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients," in *ICLR*, 2021.

[31] L. Biggio, T. Bendinelli, A. Neitz, A. Lucchi, and G. Parascandolo, "Neural symbolic regression that scales," in *ICML*, 2021, pp. 936–945.

[32] S.-M. Udrescu and M. Tegmark, "Ai feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 6, no. 16, p. eaay2631, 2020.

[33] P.-A. Kamienny, S. d'Ascoli, G. Lample, and F. Charton, "End-to-end symbolic regression with transformers," in *NeurIPS*, 2022, pp. 10 269–10 281.

[34] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search," in *International Conference on Computers and Games*. Springer, 2006, pp. 72–83.

[35] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *ECML*. Springer, 2006, pp. 282–293.

[36] D. R. White, S. Yoo, and J. Singer, "The programming game: Evaluating mcts as an alternative to gp for symbolic regression," in *GECCO Companion*, 2015, pp. 1521–1522.

[37] T. Cazenave, "Monte-carlo expression discovery," *International Journal on Artificial Intelligence Tools*, vol. 22, no. 01, p. 1250035, 2013.

[38] F. Sun, Y. Liu, J.-X. Wang, and H. Sun, "Symbolic physics learner: Discovering governing equations via monte carlo tree search," *arXiv preprint arXiv:2205.13134*, 2022.

[39] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[40] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton *et al.*, "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[41] Q. Lu, F. Tao, S. Zhou *et al.*, "Incorporating actor-critic in monte carlo tree search for symbolic regression," *Neural Computing and Applications*, vol. 33, pp. 8495–8511, 2021.

[42] R. I. McKay, N. X. Hoai, P. A. Whigham, Y. Shan, and M. O'Neill, "Grammar-based genetic programming: a survey," in *Genetic Programming and Evolvable Machines*, vol. 11. Springer, 2010, pp. 365–396.

[43] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato, "Grammar variational autoencoder," in *ICML*, 2017, pp. 1945–1954.

[44] K. Ellis, C. Wong, M. Nye, M. Sable-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum, "Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning," *PLDI*, pp. 835–850, 2021.

[45] B. Goodman and S. Flaxman, "European union regulations on algorithmic decision-making and a "right to explanation"," *AI Magazine*, vol. 38, no. 3, pp. 50–57, 2017.

[46] P. Weber, K. V. Carl, and O. Hinz, "A comprehensive review on financial explainable ai," *Artificial Intelligence Review*, vol. 57, pp. 1–40, 2024.

[47] M. T. Ribeiro, S. Singh, and C. Guestrin, ""why should i trust you?": Explaining the predictions of any classifier," in *KDD*, 2016, pp. 1135–1144.

[48] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *NeurIPS*, 2017, pp. 4765–4774.

[49] P. Bracke, A. Datta, C. Jung, and S. Sen, "Machine learning explainability in finance: An application to default risk analysis," *Bank of England Staff Working Paper*, no. 816, 2019.

[50] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decision Support Systems*, vol. 50, no. 3, pp. 602–613, 2011.

[51] Y. Chen *et al.*, "Interpretable machine learning for imbalanced credit scoring datasets," *European Journal of Operational Research*, vol. 297, no. 1, pp. 239–253, 2022.

[52] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling lime and shap: Adversarial attacks on post hoc explanation methods," in *AIES*, 2020, pp. 180–186.

[53] S. Jain and B. C. Wallace, "Attention is not explanation," in *NAACL-HLT*, 2019, pp. 3543–3556.

[54] S. Serrano and N. A. Smith, "Is attention interpretable?" in *ACL*, 2019, pp. 2931–2951.

[55] S. Wiegreffe and Y. Pinter, "Attention is not not explanation," in *EMNLP-IJCNLP*, 2019, pp. 11–20.

[56] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[57] H. Deng, "Interpreting tree ensembles with intrees," vol. 7. Springer, 2019, pp. 277–287.

[58] D. Alvarez-Melis and T. S. Jaakkola, "Towards robust interpretability with self-explaining neural networks," in *NeurIPS*, 2018, pp. 7775–7784.

[59] Z. Kakushadze, "101 formulaic alphas," *Wilmott*, vol. 2016, no. 84, pp. 72–81, 2016.

[60] T. Chen, Y. Gao, and J. Wang, "Alpha discovery via genetic programming," *arXiv preprint arXiv:2004.02311*, 2020.

[61] W. Ren *et al.*, "Alpha mining and enhancing via warm start genetic programming for quantitative investment," *arXiv preprint arXiv:2412.00896*, 2024.

[62] S. Yu *et al.*, "Alphagen: Automate alpha factor generation with reinforcement learning," *arXiv preprint arXiv:2306.12964*, 2023.

[63] Z. Zhang and S. Zohren, "Deep learning for alpha generation," *The Journal of Financial Data Science*, vol. 2, no. 3, pp. 25–39, 2020.

[64] H. Xu *et al.*, "Alphaforge: A framework to mine and dynamically combine formulaic alpha factors," *arXiv preprint arXiv:2406.18394*, 2024.

[65] Q. Zhao *et al.*, "Alpha-gpt: Human-ai interactive alpha mining for quantitative investment," *arXiv preprint arXiv:2308.00016*, 2024.

[66] R. Cont, "Empirical properties of asset returns: stylized facts and statistical issues," *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.